4. Multimodal Visual Understanding + SLAM Navigation (Text Version)

1. Course Content

1. Learn how the large model closes a cross-modal, long-chain loop involving multi-point navigation, physical-space semantic mapping, and multimodal visual perception;
2. Master the waypoint calibration of the map semantic mapping file `map_mapping.yaml` and the Dify variable synchronization mechanism;
3. Learn to perform initial pose calibration (2D Pose Estimate) in the RViz2 visualization interface;
4. Issue complex long-duration tasks through the text terminal: cross-room navigation patrol, image-perception multimodal memory, returning along the original route, and outputting a summary report;
5. Analyze the core source code of the collaborative scheduling between the underlying navigation Action client and the action server.

---

2. Preparation

2.1 Content Description

> This lesson applies to controllers such as Raspberry Pi 5, Jetson Orin NANO, and D-Robotics RDK X5:
>
> - **Raspberry Pi 5**: You must first enter the Docker container before running ROS 2 related commands. Refer to [3.Enter Docker (For Raspberry Pi)] in the [Robot Configuration and Operation Guide];
> - **Orin / RDK X5**: Run commands directly in the onboard terminal; there is no need to enter Docker.

2.2 Start the Local Dify Large-Model Environment

Enter the following command in the host terminal to start the Dify service:

```
sh ~/bringup_dify.sh
```

The terminal prints the following message, indicating that the Dify service is connected successfully:



> **Important**
>
> **Precondition**:
> This lesson requires the robot to navigate on an already-built map. Please make sure that a grid map (such as `map.yaml`) has been built and saved according to the navigation chapter tutorial.

2.3 Configure the Map Waypoint Semantic Mapping (map_mapping.yaml)

To let the large model understand the physical coordinates of natural-language places such as "tea room," "kitchen," and "office," a mapping relationship between place names and absolute map coordinates must be established in the configuration file.

1. Calibrate Room Coordinates and Orientation

Connect to the robot desktop via VNC and run the following command to start the navigation node:

- Start the low-level layer

```
ros2 launch yahboomcar_nav laser_bringup_launch.py
```

- Visualization node, or alternatively start the display on a virtual machine

```
ros2 launch yahboomcar_nav display_nav_launch.py
```

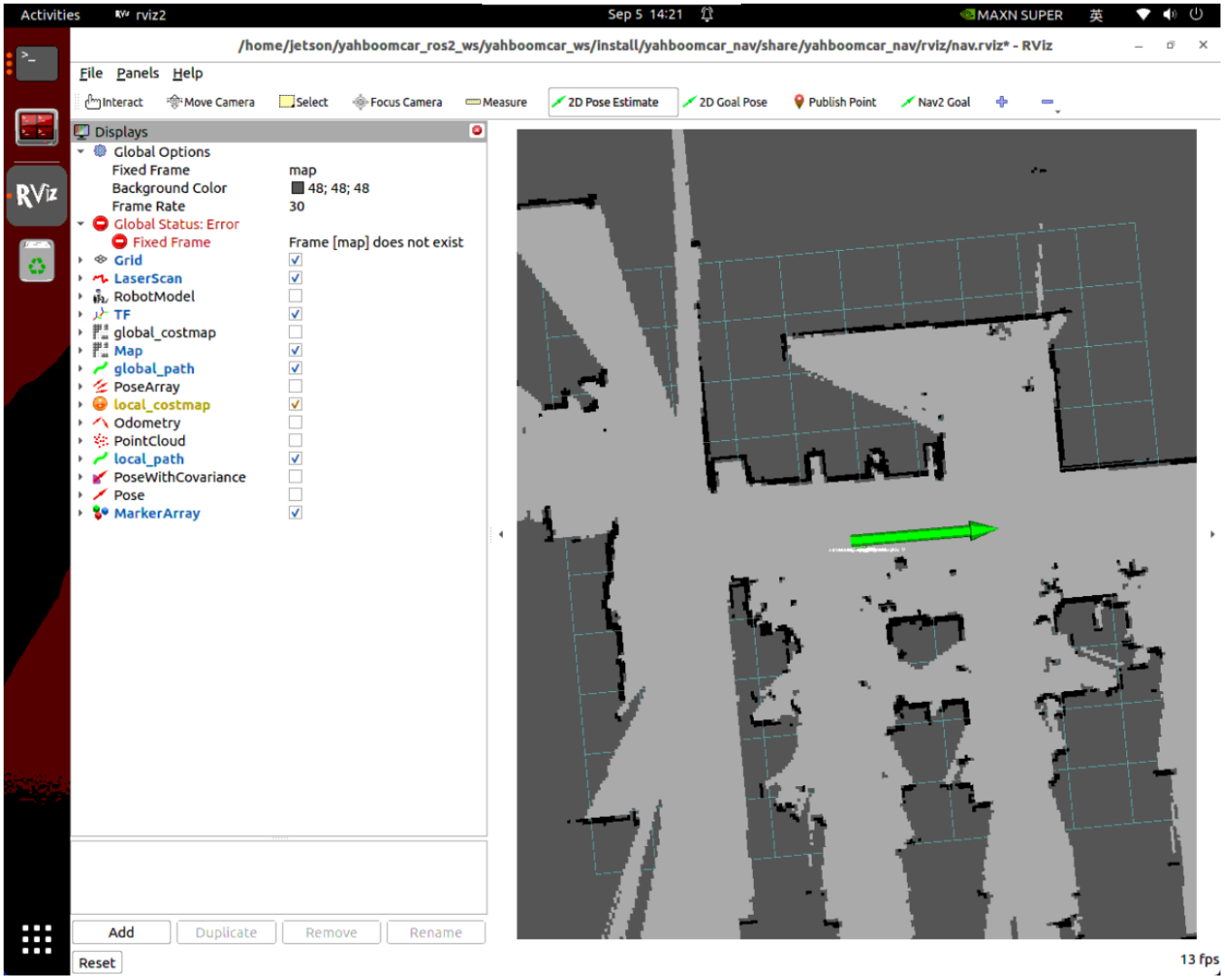
- Start navigation. **Note: whichever localization method was used during waypoint calibration must be used for subsequent navigation, because the poses of different localization methods may differ**

```
# AMCL localization + navigation
ros2 launch yahboomcar_nav navigation_teb_launch.py

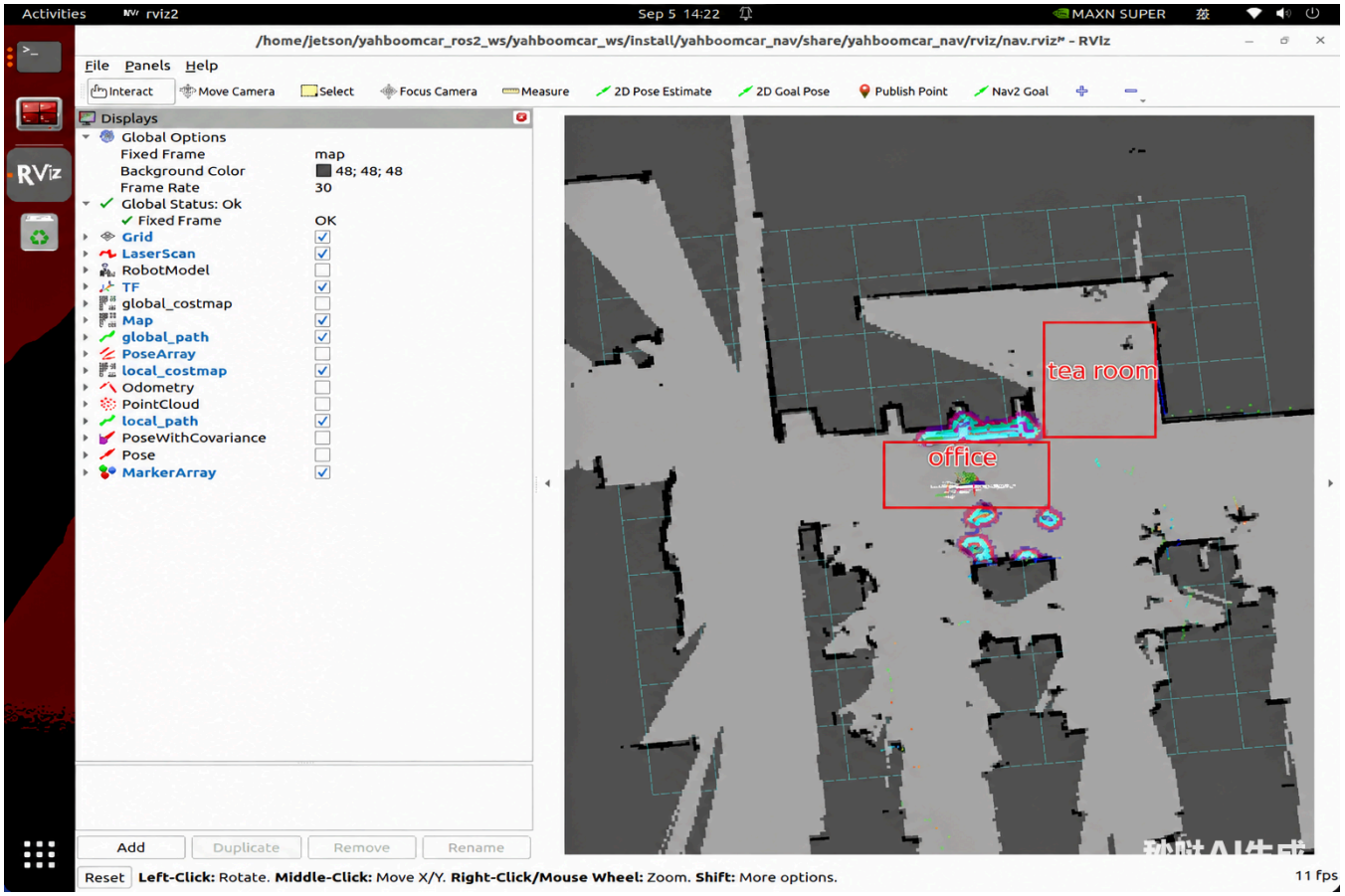
# Relocalization method; see the Cartographer relocalization navigation for details
ros2 launch yahboomcar_nav localization_imu_odom.launch.py use_rviz:=false
load_state_filename:=$HOME/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/maps/map.pbst
ream

ros2 launch yahboomcar_nav navigation_cartodwb_launch.py
maps:=$HOME/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/maps/map.yaml
```

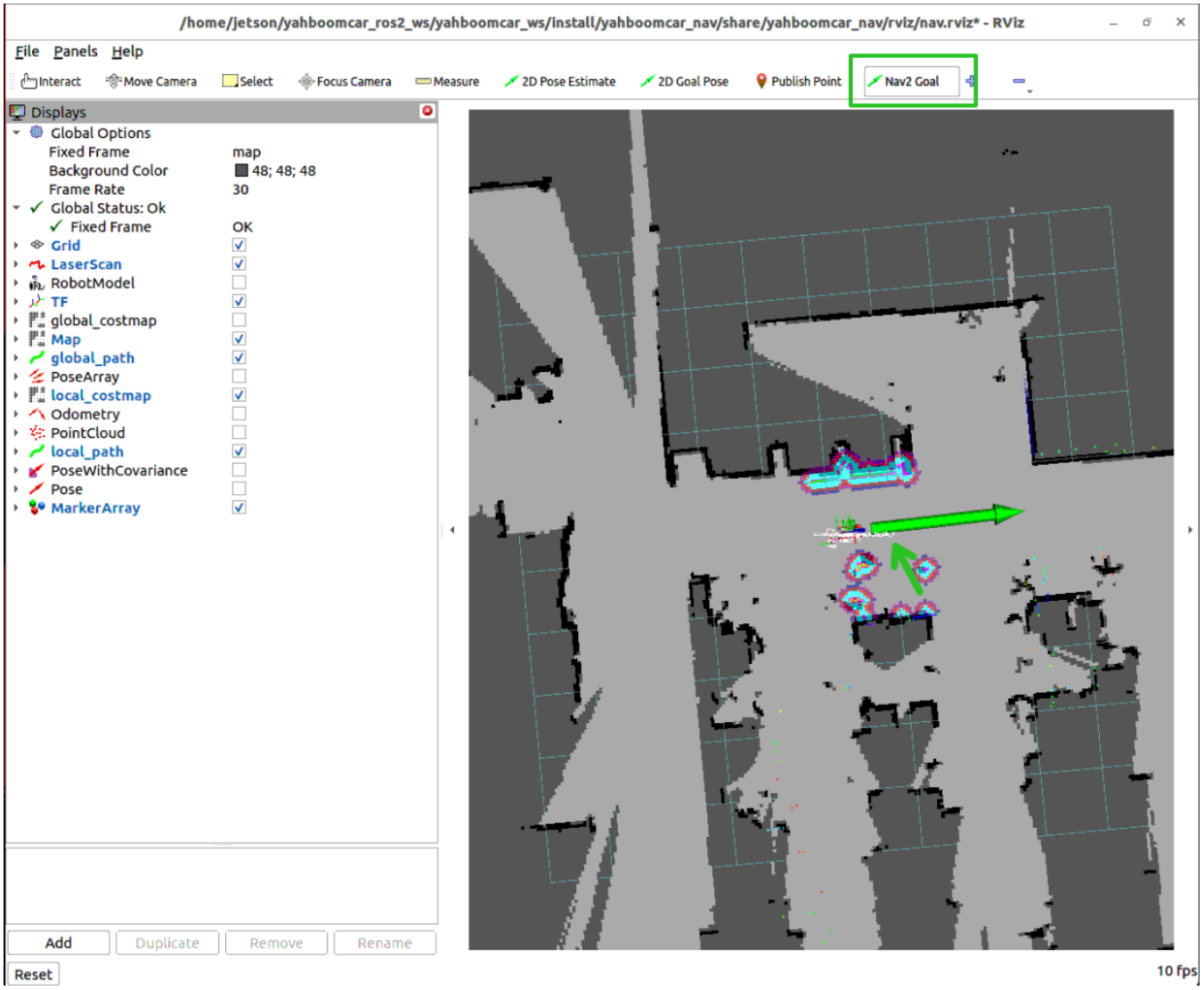
Open the RViz2 navigation interface (see the startup steps below), click the **2D Pose Estimate** tool in the upper toolbar, and mark the robot's current real physical position and orientation on the map:



Combined with the target areas of the robot in the map (such as "office," "tea room"):



In RViz2, use the **2D Goal POSE** tool to click the target position of the room, driving the robot to navigate to that point:



After the robot arrives at the target room, run the following command in the terminal to obtain the robot's precise pose parameters in the current `map` coordinate system:

```
ros2 run tf2_ros tf2_echo map base_footprint
```

The terminal prints the translation `Translation [x, y, z]` and quaternion rotation `Rotation [x, y, z, w]` of the current pose:

```
jetson@yahboom:~/yahboomcar_ros2_ws/software/library_ws$ ros2 run tf2_ros tf2_echo map base_footprint
[INFO] [1788590003.613868389] [tf2_echo]: Waiting for transform map ->  base_footprint: Invalid frame ID "map" passed to canTransform argument target_fram
e does not exist
At time 1788590005.493515420
- Translation: [0.206, -0.598, 0.000]
- Rotation: in Quaternion (xyzw) [0.000, 0.000, -0.737, 0.676]
- Rotation: in RPY (radian) [0.000, 0.000, -1.656]
- Rotation: in RPY (degree) [0.000, 0.000, -94.883]
- Matrix:
 -0.085  0.996  0.000  0.206
 -0.996 -0.085 -0.000 -0.598
 -0.000  0.000  1.000  0.000
  0.000  0.000  0.000  1.000
At time 1788590006.477053138
- Translation: [0.206, -0.598, 0.000]
- Rotation: in Quaternion (xyzw) [0.000, 0.000, -0.737, 0.676]
- Rotation: in RPY (radian) [0.000, 0.000, -1.656]
- Rotation: in RPY (degree) [0.000, 0.000, -94.883]
- Matrix:
 -0.085  0.996  0.000  0.206
 -0.996 -0.085 -0.000 -0.598
 -0.000  0.000  1.000  0.000
  0.000  0.000  0.000  1.000
```

2. Edit the map_mapping.yaml File

The configuration file path for each controller board is as follows:

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/multi_brains/config/map_mapping.yaml
```

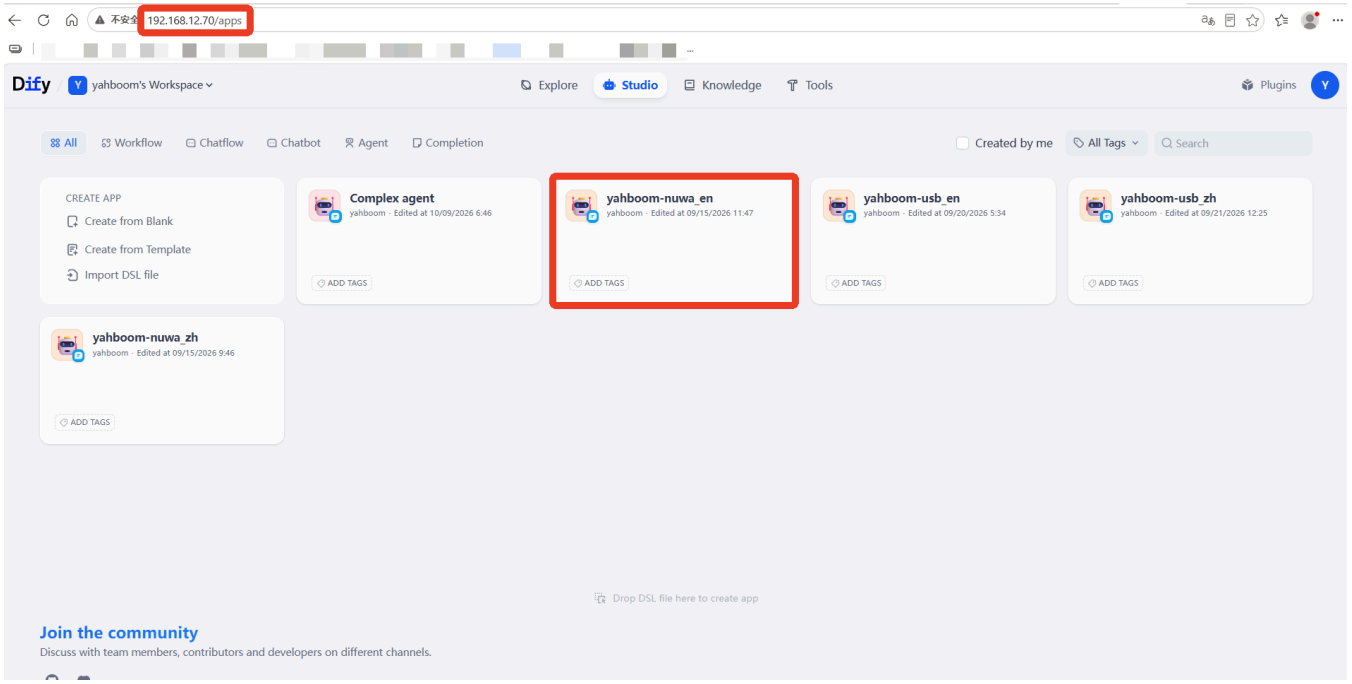
Open the `map_mapping.yaml` map mapping file (by any one of VNC, VSCode, command line, etc.), and fill in the room names and coordinates obtained above under `common_map_areas`:

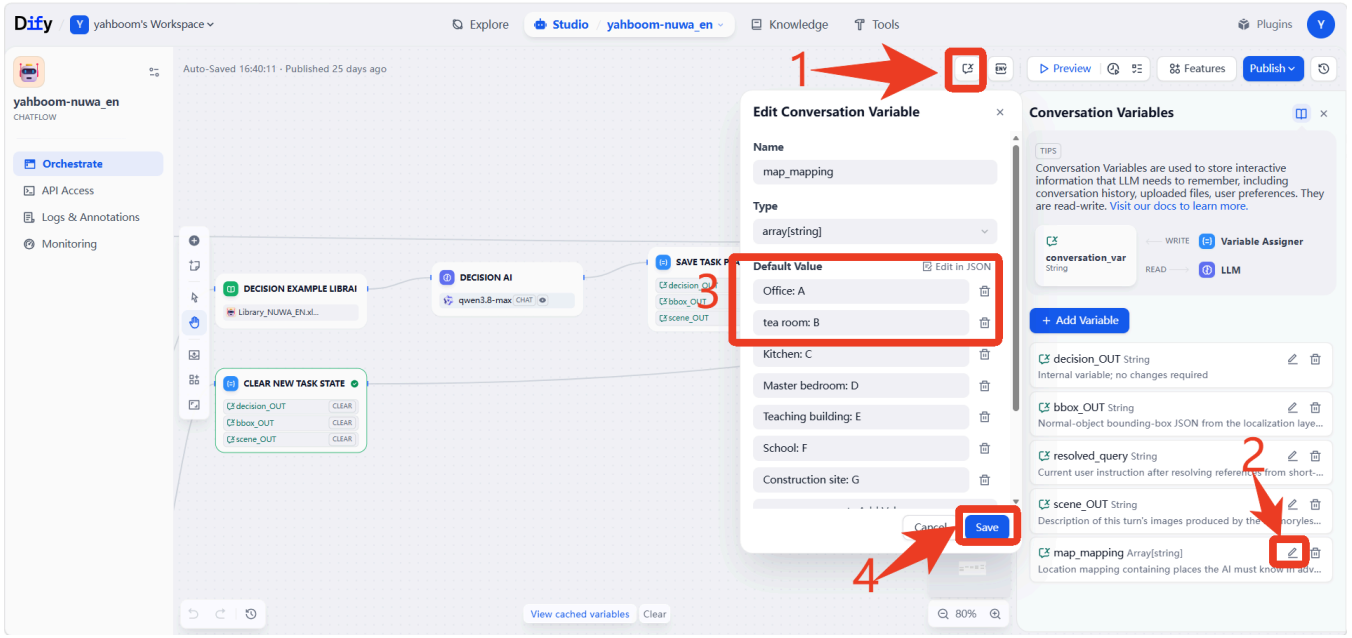
```yaml
common_map_areas: # common navigation
  A:
    name: 'Office'
    position:
      x: 0.206
      y: -0.598
      z: 0.0
    orientation:
      x: 0.0
      y: 0.0
      z: -0.737
      w: 0.676
  B:
    name: 'tea room'
    position:
      x: 1.064
      y: -2.432
      z: 0.0
    orientation:
      x: 0.0
      y: 0.0
      z: -0.141
      w: 0.990
```

After completing the configuration, rebuild and source the environment variables:

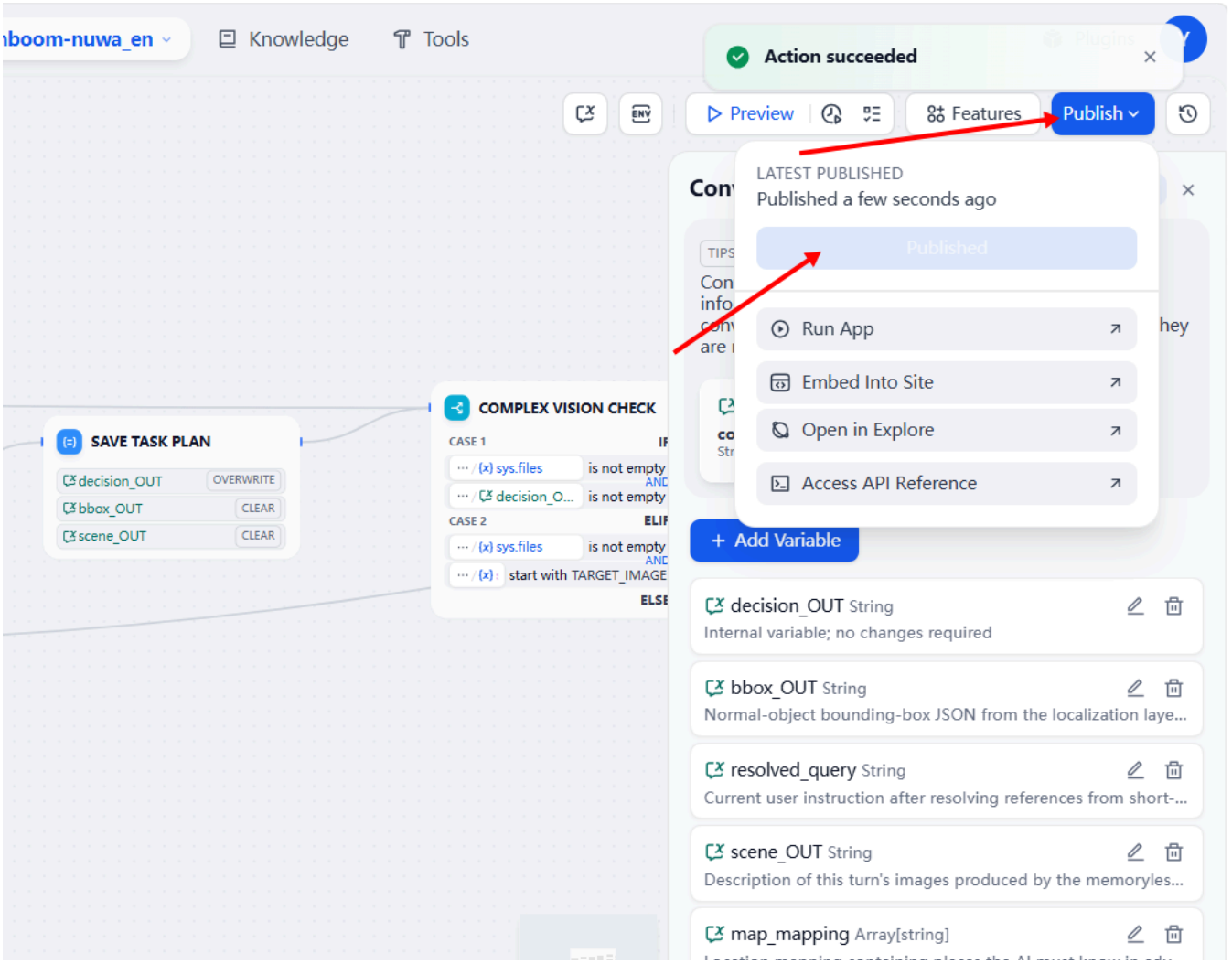
```
cd ~/yahboomcar_ros2_ws/yahboomcar_ws/
colcon build --packages-select multi_brains
source install/setup.bash
```

Then open the Dify workbench application in a browser (`vehicle IP`), synchronously update the `map_mapping` variable in the [Conversation Variables] of the corresponding application, and click [Publish Update]:





Click Publish Update

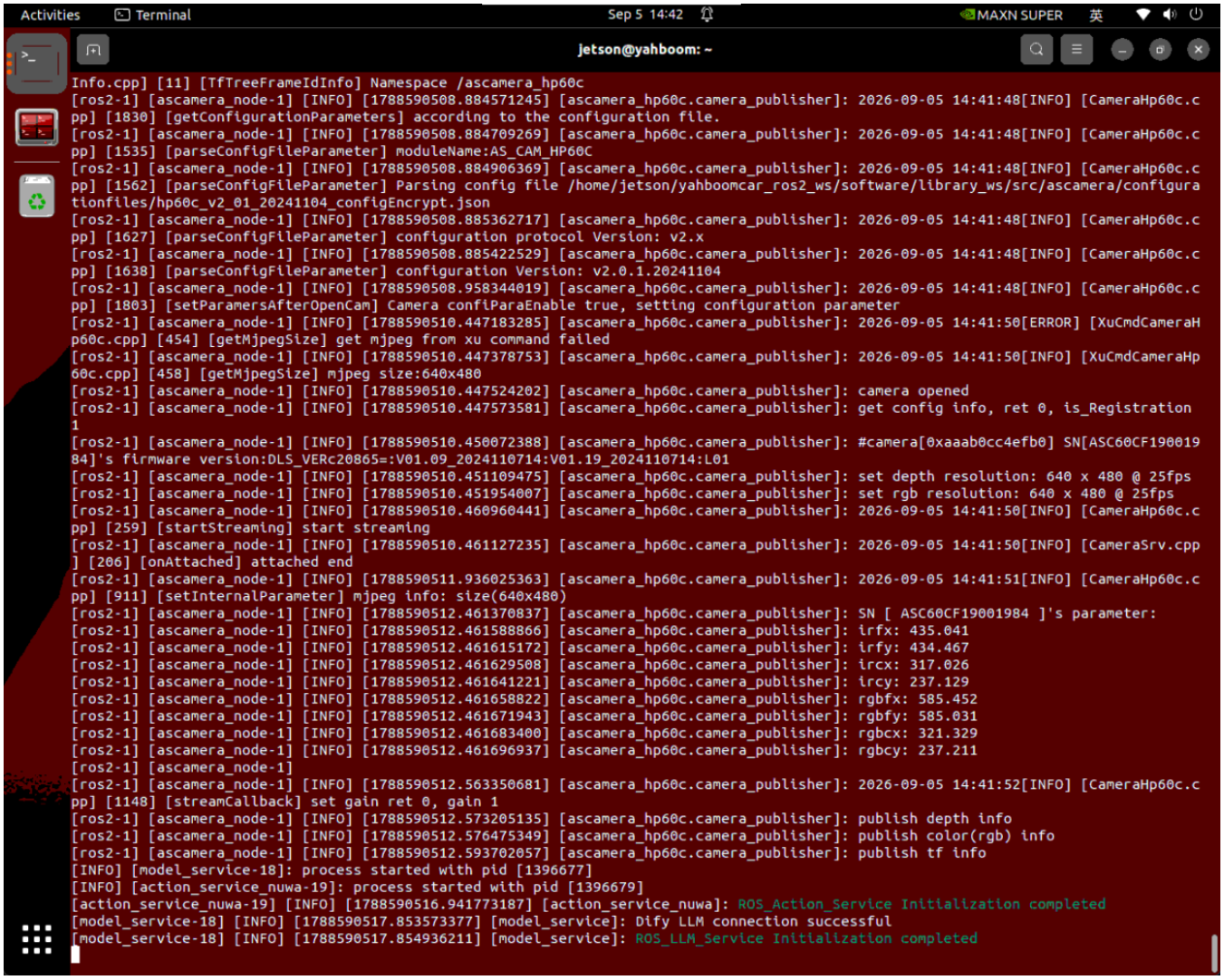


3. Running Examples

3.1 Start the System Master Control Node (Text Interaction Mode)

After closing all terminals, re-enter the following command in a terminal to start the large-model embodied control service:

```
ros2 launch multi_brains llm_agent_control.launch.py text_chat_mode:=True
```



3.2 Start the SLAM Navigation and Visualization System

On the onboard desktop or a virtual machine, open another terminal and start the navigation visualization interface:

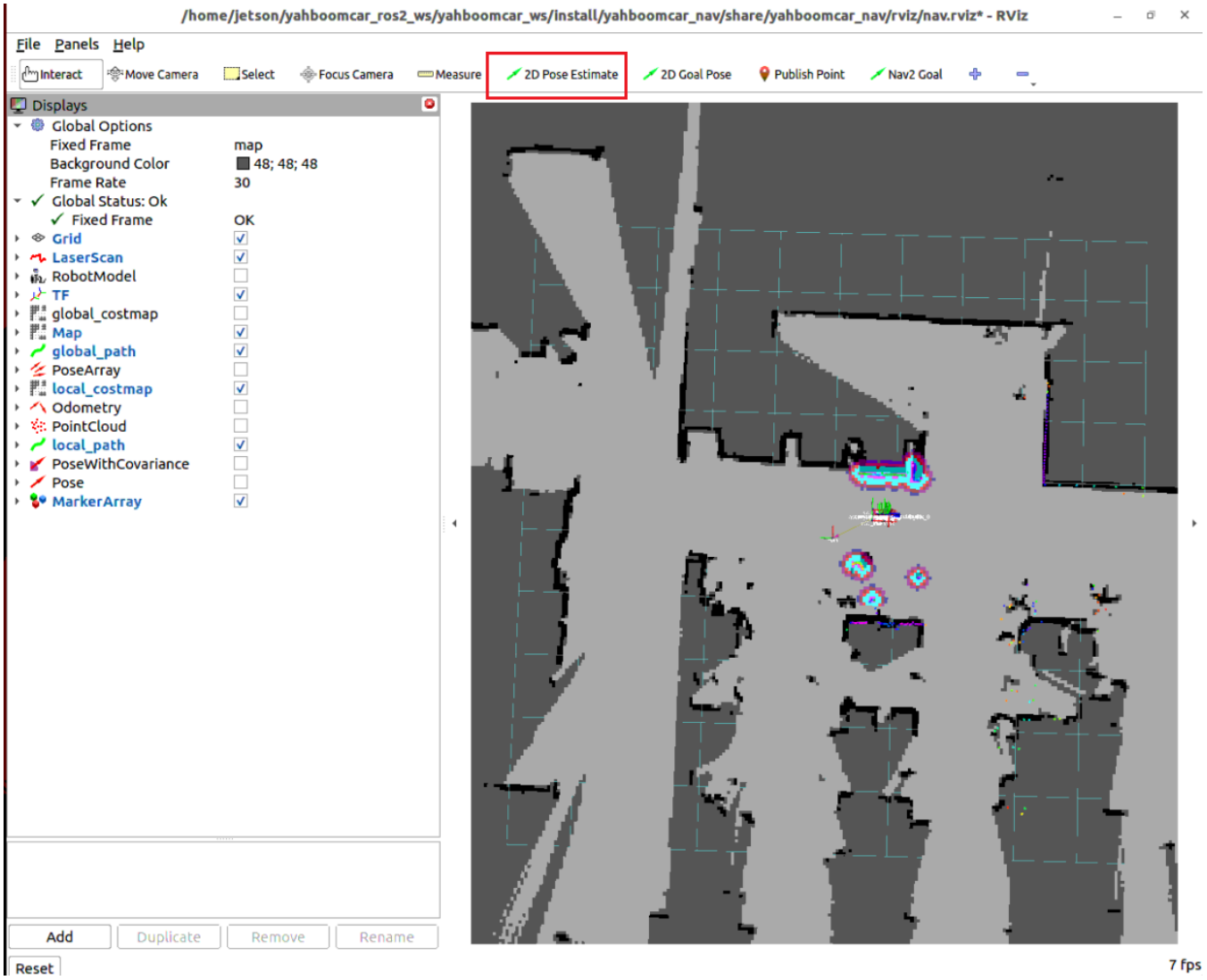
```
ros2 launch yahboomcar_nav display_nav_launch.py
```

Then open another terminal and start the underlying navigation algorithm (the TEB local path planner is recommended): **Note: whichever localization method was used during waypoint calibration must be used for subsequent navigation, because the poses of different localization methods may differ**

```
ros2 launch yahboomcar_nav navigation_teb_launch.py
# Relocalization method; see the Cartographer relocalization navigation for details
ros2 launch yahboomcar_nav localization_imu_odom.launch.py use_rviz:=false
load_state_filename:=$HOME/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/maps/map.pbstream

ros2 launch yahboomcar_nav navigation_cartodwb_launch.py
maps:=$HOME/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/maps/map.yaml
```

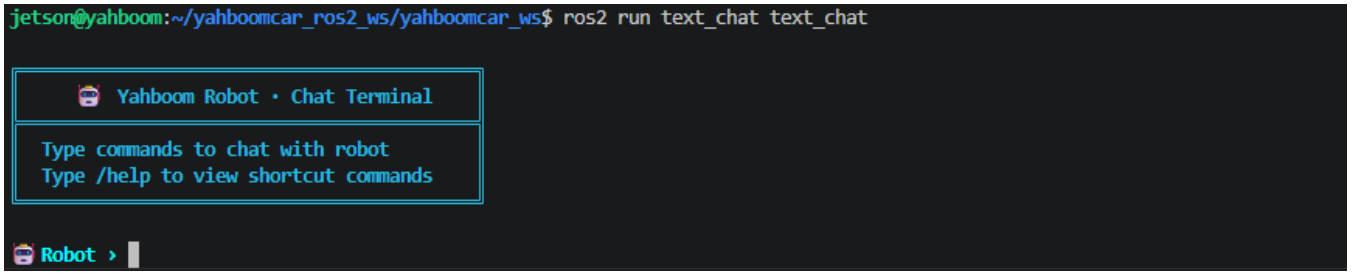
In the opened RViz2 window, use the **2D Pose Estimate** tool to perform initial alignment and localization according to the robot's parked position in the real scene.



3.3 Start the Text Interaction Terminal

Enter the following command in the SSH terminal on the computer to start the text interaction interface:

```
ros2 run text_chat text_chat
```



3.4 Classic Test Case (Cross-Room Patrol and Multimodal Long-Term Memory Report)

Enter the composite inspection task statement in the text interaction terminal:

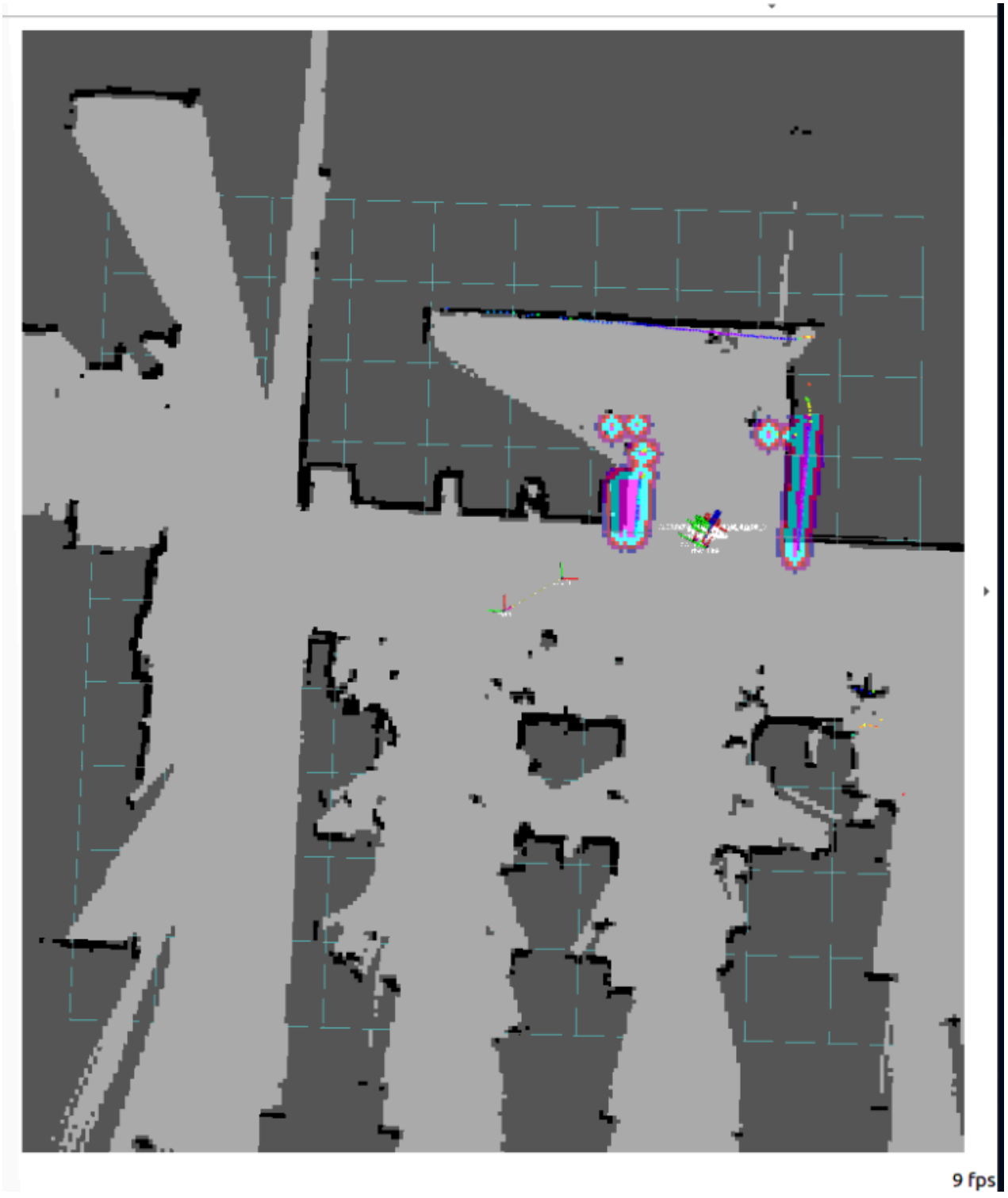
```
Go to the tea room, help me see what items are inside, then return to the office and report.
```

The terminal prints the continuous long-chain task plan parsed by the large model:

```
[INFO] [action_service_nuwa-19]: process started with pid [1396679]
[action_service_nuwa-19] [INFO] [1788590516.941773187] [action_service_nuwa]: ROS_Action_Service Initialization completed
[model_service-18] [INFO] [1788590517.853573377] [model_service]: Dify LLM connection successful
[model_service-18] [INFO] [1788590517.854936211] [model_service]: ROS LLM Service Initialization completed
[model_service-18] [INFO] [1788590624.086900889] [model_service]: 决策层规划: 1.前往茶水间 2.获取当前图像 3.回到办公室
[model_service-18] [INFO] [1788590624.087679045] [model_service]: 'action' : [ 'navigation(B)' ], 'response' : 好的，我现在前往茶水间。
[action_service_nuwa-19] [INFO] [1788590624.108792667] [action_service_nuwa]: Recorded Pose: Position: x=0.38436378080487593, y=-0.7
94306183586345,z=0.0, Orientation: x=0.0,y=0.0, z=-0.7096476982053671, w=0.7045567006507171
```

Execution Process and Behavior:

1. **Head to the master bedroom**: the robot chassis automatically plans an obstacle-avoidance path and navigates to the "master bedroom" waypoint:

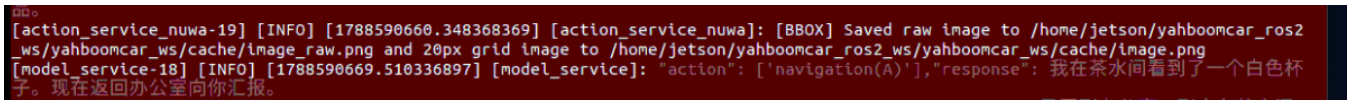


2. **Visual perception and memory**: after arriving at the master bedroom, the robot brakes to a stop and

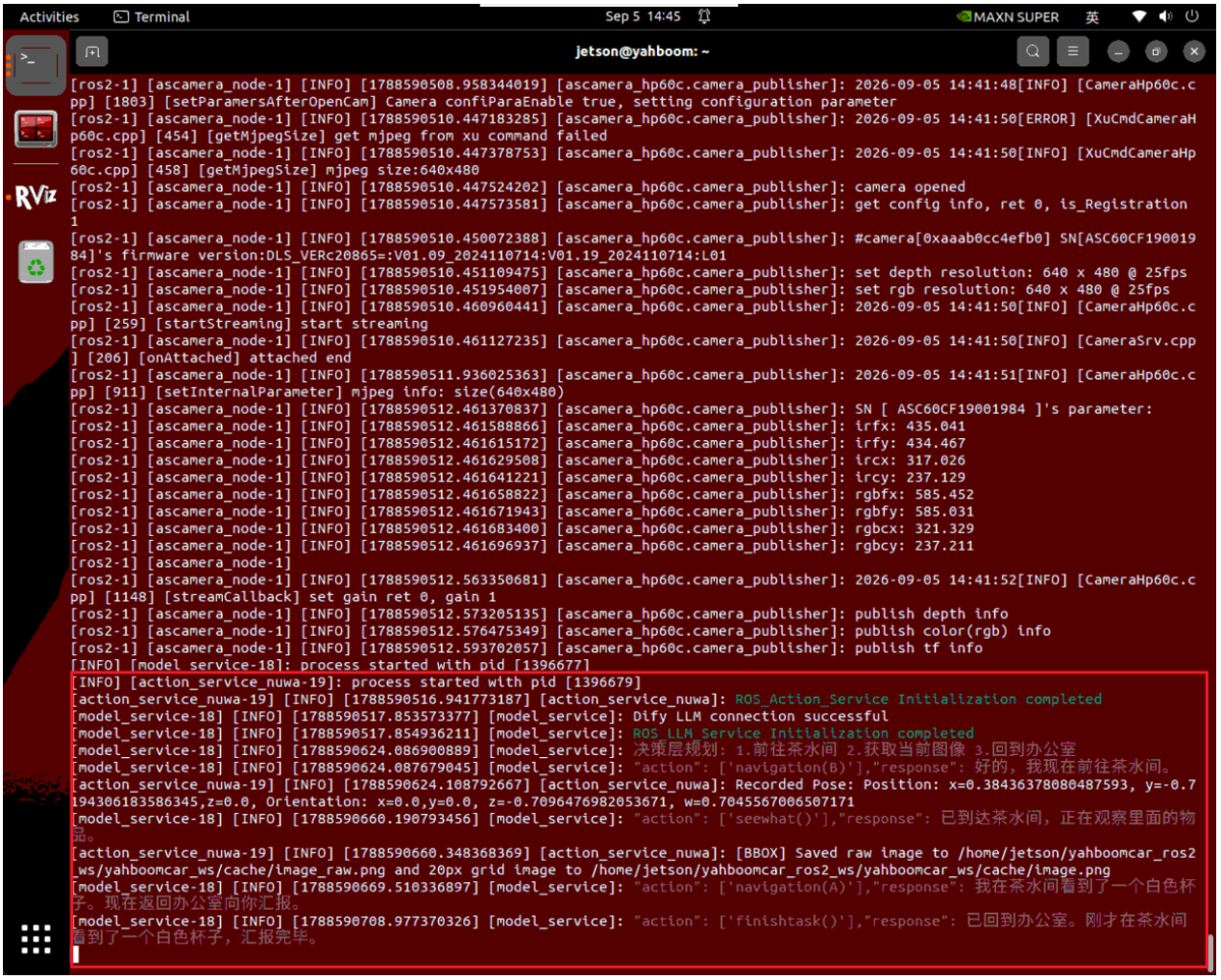
calls the `seewhat()` method to wake up the camera to capture a photo of the current scene; the multimodal large model analyzes and recognizes the items in the scene (such as a tissue box, a water cup, and blocks):

```
~/yahboomcar_ros2_ws/yahboomcar_ws/cache/image_raw.png
```

3. **Return to the kitchen**: the large model keeps the list of observed objects in the session context,



4. **Proactive text report**: after arriving at the kitchen, the robot triggers `finish_dialogue()` and gives the user a detailed summary report in the text interaction terminal:



4. Core Source Code Analysis

Source package path:
`~/yahboomcar_ros2_ws/yahboomcar_ws/src/multi_brains`

4.1 Semantic Waypoint Parsing and Navigation Goal Issuance (action_service_*.py)

In `ActionController`, the `navigation` method encapsulates the Nav2 `NavigateToPose` action client:

```
def navigation(self, point_name):
    # 1. Look up the pose parameters of the corresponding waypoint in the map mapping table
    target_info = self.map_areas.get(point_name)
    if not target_info:
        self.get_logger().error(f"Target waypoint not found: {point_name}")
        return False

    # 2. Assemble the PoseStamped goal message
    goal_pose = PoseStamped()
    goal_pose.header.frame_id = "map"
    goal_pose.header.stamp = self.get_clock().now().to_msg()
    goal_pose.pose.position.x = target_info['position']['x']
    goal_pose.pose.position.y = target_info['position']['y']
    goal_pose.pose.orientation.z = target_info['orientation']['z']
    goal_pose.pose.orientation.w = target_info['orientation']['w']

    # 3. Send the goal asynchronously and block waiting for the navigation completion
feedback
    nav_goal = NavigateToPose.Goal()
    nav_goal.pose = goal_pose
    self.nav_client.send_goal_async(nav_goal)
    ...
```

4.2 Long-Chain Cross-Room Context Preservation Mechanism (model_service.py)

1. **Chain-of-thought orchestration**: after receiving the composite command, the large model decomposes the task into a standard action flow:

   ```
   [
     {"action": "navigation", "params": {"point_name": "master bedroom"}},
     {"action": "seewhat", "params": {}},
     {"action": "navigation", "params": {"point_name": "kitchen"}},
     {"action": "finish_dialogue", "params": {"text": "report content..."}}
   ]
   ```

2. **State-machine safety isolation**: `model_service.py` maintains a session lock, issuing the next step only after each underlying action is completed and fed back through the ROS 2 Action. This ensures that the robot captures a photo only after it has fully reached the master bedroom and come to a stop, and starts the return trip only after the visual analysis is complete, guaranteeing the high robustness of long-chain embodied tasks.